

Algorithms Foundations & Core Data Structures

Complete Handwritten Lecture Notes & Solved Problems

Part I: Algorithms Foundations & Core Data Structures

Linear lists, stacks, queues, sparse matrices, binary trees, threaded binary trees, Huffman coding, heaps, and sorting algorithms.

PAGE 1

The note discusses data structures and algorithms. It mentions:

- Linear and non-linear data structures
- Tables, trees, and graphs
- Storage structures: sequential, linked, indexed, and associative arrays
- Operations: search, insert, delete, merge, split, and traversal
- Pattern matching
- Sorting algorithms: bubble sort, quick sort, insertion sort, direct insertion, binary insertion, shell sort, selection sort, merge sort, and heap sort.

Chapter 1. Introduction

1.1 Data Abstraction and Binary Relations

- **Data Abstraction:**
- Elements + Relations $\rightarrow$ Directed Acyclic Graphs
- Cartesian Product of Sets

$$A \times B = \{(a, b) \mid a \in A \text{ and } b \in B\}$$

- Extension Concept: n Sets Cartesian Product

- **Binary Relations:**

- A subset of $A \times B$. $A = B$ is called a binary relation R .

- $A \times A \supset R = \{(a, b) \mid a \in A \text{ and } b \in A \text{ and satisfy certain conditions}\}$

- Reflexivity: $\forall a \in A, (a, a) \in R$; Anti-reflexivity: $\forall a \in A, (a, a) \notin R$.

- Symmetry: $(a, b) \in R$ then $(b, a) \in R$; Anti-symmetry: $(a, b) \in R$ and $a \neq b$, then $(b, a) \notin R$.

- Transitivity: $(a, b) \in R$ and $(b, c) \in R$, then $(a, c) \in R$. The relation R is transitive.

- Note: Reflexivity and symmetry + anti-symmetry $\rightarrow$ anti-symmetry: Counterexample.

- Equivalence Relation: Reflexivity, symmetry, transitivity $\rightarrow$ equivalence class $\rightarrow$ partition of the set.

- Partial Order Relation: Reflexivity, anti-symmetry, transitivity: cannot determine whether any element $(a, b) \in R$ (ordering)

- Total Order Relation: $\forall a, b \in A$, either $(a, b) \in R$ or $(b, a) \in R$, where R is a partial order relation $\rightarrow$ total order.

- Strict Partial Order Relation (Total Order Relation): Reflexivity, anti-symmetry, transitivity: e.g., $\in$.

1.2 Data Structure Basic Concepts

- **Data Logical Structure:**

- $B = (D, R)$ where D is a set and R is a relation on D . Linear: list; Non-linear: tree, graph.

- **Data Storage Structure:**

- Sequential, Linked, Index, Array.

1.3 Algorithm Description and Analysis

- Algorithm Basic Characteristics
- Algorithm Description
- Algorithm Design Common Methods

Brute Force Method: Find the maximum element in A, Knapsack, string matching...

Divide and Conquer Method: Find the maximum element in A, Merge Sort, Quick Sort, Shell Sort...

Greedy Method: Find the maximum element in A, Knapsack, Binary Search, B-tree search...

Backtracking Method: Maze, Knapsack...

Dynamic Programming Method: Floyd, Warshall...

Time Complexity and Space Complexity:

Time: Best/Worst, addition/multiplication; loop/recursion

Chapter 2: Linear List Storage and Operations

2.1 Linear List Concepts

- $B=(D,R)$ $D=\{a_i \mid i=1,2,\dots,n\}$ $R=\{a_i, a_{i+1} \mid i=1,2,\dots,n-1\}$

2.2 Linear List Operations

- Consider the insertion and deletion of elements in an ordered list.
- Limitation: Requires contiguous storage space; insertion and deletion efficiency.

2.3 Stack

- Push upward: $\text{top} = -1$; Pop downward: $\text{top} = \text{ms}$.
- Double Stack: Efficiently handles stack operations.

if (P is small enough) // Directly solve P;

else

for ($i=1$; $i \leq k$; $i++$) $y_i = \text{Divide}(P_i)$ // Solve sub-problems

Stack Work: Push: push; Pop: pop; Save local variables, actual parameters, return address.

2.4 Queue

Linear Queue:

$$a_1, a_2, \dots, a_n \rightarrow i$$

Circular Queue:

0 element at $m - 1$ element after. Clear: delete $\text{front} = \text{rear} = 0$

Queue Empty:

$$\text{front} = \text{rear}$$

Queue Full:

$$(\text{rear} + 1) \% m = \text{front}$$

Priority Queue:

{New element in queue tail, dequeue highest priority element.}

2.5 Arrays and Matrix Representation

1. Array Sequential Allocation

Row Major:

$$AD(a_{ij}) = AD(a_{11}) + [(i - 1)n + j - 1] \cdot \text{unit}; AD(a_{ij}) = AD(a_{11}) + (in + j) \cdot \text{unit}$$

Column Major:

$$AD(a_{ij}) = AD(a_{11}) + [(j - 1)n + i - 1] \cdot \text{unit};$$

2. Rule of Matrix Storage

$$\text{Row: } k = \frac{1}{2}i(i-1) + j$$

$$\text{Column: } k = \frac{(2n - i - j)}{2} + i - (j - 1)$$

Row: $k = i(i - 1)/2 + j, i > j$

Column: $k = (j - 1)/2 + i, i < j$

Row: eg : Triangular $k = 5(i - 1) - 3 + [j - (i - 3)], a_{ij} =$

$$k = \left\{ \begin{array}{ll} \end{array} \right.$$
$$1 \ \& \ \text{\texttt{\text{if}}} \ i < j \ \text{\texttt{\text{or}}} \ i > j \ \text{\texttt{\text{and}}} \ i \leq j + \frac{1}{2} \ \backslash$$
$$0 \ \& \ \backslash \text{text}\{\text{else}\}$$
 $\end{array} \right.\$$

3. Sparse Matrix in 3D Array Representation - 3D Array Sequential Table B(row, col, value)

$$\{\text{pos}[i, j] : \text{Sparse matrix first non-zero element in B, pos}[i, j] = \text{pos}[i - 1] + \text{num}[i - 1]$$

num[i,j]: Sparse matrix non-zero element count. num[B[i,j]] = num[B[i,j]] + 1

Chapter 3: Linked Lists

3.1 Linear List's Linked Storage

- Linear List Concept

The reason for using a linked list structure: the length is greater than the continuous space; the length cannot be determined in advance; the length changes frequently.

- Linear List and Its Structure

Insertion special cases: empty list; insert at the first position; cannot find the insertion position.

Deletion special cases: empty list; delete the first element; cannot find the deletion position.

Merge operation: select basic link. $O(m+n)$

Split operation (based on odd/even): $O(n)$.

3.2 Linked Stack and Linked Queue

- Linked Stack

top $\rightarrow$ [] $\rightarrow$ [] $\rightarrow$... $\rightarrow$ []

Empty stack using linked stack storage (static)

- Linked Queue

[] $\rightarrow$ [] $\rightarrow$... $\rightarrow$ []

front $\rightarrow$ rear

Empty queue similarly stored (static)

3.3 Circular Linked List

$H \rightarrow [] \rightarrow [a_1] \rightarrow [a_2] \rightarrow [a_3] \rightarrow \dots \rightarrow [a_n] \rightarrow H$

Empty: $H \rightarrow []$

Advantages: empty list does not need special handling; determine the tail node

condition: $p \rightarrow \text{next} == \text{head}$.

Merge operation: not considering empty list; when one chain is complete and the other chain still has elements, find the tail node of the other chain and connect it to the new chain.

Split operation: new split head and tail points, based on odd/even, choose basic chain.

3.4. Multilink List

3.5. Generalized Table

Generalized Table Concepts:

- Recursive structure; recursive definition; table depth = number of nested layers = maximum depth of table nodes. Table length = next number

Generalized Table Storage Structure:

- head() tail() (first layer)
- **Additional Table Head Node:**
 - flag = 0
 - sublist
 - next
- **Atomic Node:**
 - flag = 1
 - data
 - next
- **Subtable Node:**
 - flag = 2
 - sublist
 - next

Chapter 4: Trees and Binary Trees

4.1. Tree Basic Concepts:

- Root Node (root)
- Node (node)
- Node degree
- Subtree number
- Tree degree: maximum node degree
- From the root node, the levels are 1, 2, ..., the tree's depth is the maximum level.
- Properties:
 - n nodes, degree d_i ($i=1, 2, \dots, n$) $\Rightarrow n = \sum d_i + 1$.
 - n nodes in a binary tree, minimum depth: $\lfloor \log_2(n(k-1)+1) \rfloor$: excluding leaf nodes.

4.2. Binary Trees

Binary Tree Definition

Binary Tree Basic Properties:

- Leaf node number n_0 , degree 2 node number $n_2 \Rightarrow n_0 = n_2 + 1$.

Proof: Total node number n , degree 1 node number n_1

$$n = n_0 + n_1 + n_2.$$

Except the root node, each node has one branch connected:

$$n - 1 = 2n_2 + n_1$$

Solve:

$$n_0 = n_2 + 1.$$

2. Complete Binary Tree: Left -> Right Full; Ideal Balanced Binary Tree: Random Full; Full Binary Tree: Complete

n nodes, depth $\lceil \log_2 n \rceil + 1$

Depth h , maximum nodes $2^h - 1$

3. i's Left: $2i+1$, Right: $2i+2$, Parent: $\lfloor i-1 \rfloor / 2$

$$2i$$

$$2i + 1$$

$$\lfloor i / 2 \rfloor$$

Three. Binary Tree Storage

Binary Linked List; Complete Binary Trees can use sequential storage, like heaps.

Four. Binary Tree Operations

Preorder Traversal: Visit -> Left -> Right

Time: $O(n)$; Space: $O(\log_2 n) \sim O(n)$

Inorder Traversal: Left -> Visit -> Right

Non-recursive algorithm: Book P165; Supplement P46

Postorder Traversal: Left -> Right -> Visit

Layer Traversal: Left Enqueue -> Right Enqueue -> Dequeue: Loop while $(! \text{EmptyQueue}())$

Five. Binary Tree Counting

Inorder + Preorder or Inorder + Postorder or Inorder + Postorder => Element Counting!

Number of nodes in the corresponding inorder sequence:

$$\frac{1}{n+1} C_n^n$$

(Inorder corresponding to the preorder sequence).

From preorder and inorder to determine the binary tree method: Preorder determines the root, inorder determines the left and right subtrees.

Six. Threaded Binary Trees

Left tag: Lkid: data: Right tag. tag=0: Original; tag=1: Threaded

Preorder Threaded: Current node and preorder predecessor add thread -> Left -> Right.

Inorder Threaded: Left -> Current node and inorder predecessor add thread -> Right.

Postorder Threaded: Left -> Right -> Current node and postorder predecessor add thread.

4.3 Binary Tree Applications

- Convert Ordered Trees to Binary Trees

Ordered Tree -> Binary Linked -> Binary Tree: Tree's child-sibling representation; Single branch ordered tree does not distinguish left and right

- Huffman Binary Tree (Huffman). Height h : $2^{h-1} \leq n \leq 2^h - 1$. $\{n_0 = n_2 + 1\} \rightarrow \{n_0 = n/2\}$

Weight:

$$\sum_i^n \text{weight}(i) \cdot \text{length}(i) = \min$$

$$\{n_0 + n_2 = n\} \{n_2 = n/2\}$$

Method: Each time from the forest select weight smallest two trees to form a binary tree, root weight = weight + 1

Application: Huffman Encoding (Prefix Encoding): Left 0 Right 1.

III. Binary Search Tree (BST) Height h : $h < n < 2^h - 1$

Left < Right: In-order sequence, insert in in-order sequence. Time $O(\log n) \sim O(h)$, space: same

Delete:

- Leaf: Directly delete
- Single branch (degree 1): Delete child
- Double branch (degree 2): In-order predecessor replaces $\rightarrow$ delete in-order predecessor (single branch: ...)

IV. Heap (heap). Height h : $2^{(h-1)} \leq n \leq 2^h - 1$

Parent: Up > Down; Child: Up < Down $\Rightarrow$ Complete binary tree stored in order. Level order numbering

Recursion: Left child $2i+1$, right child $2i+2$, parent $\lfloor (i-1)/2 \rfloor$

Insert: Insert at tail (right end); for this element siftup: $O(\log n)$

Delete: Delete from tail; direct delete; delete from top; swap with tail element; sift down; clear

Build: Each element execute insert (siftup): $O(n \log n)$

Build after insert (right to left) siftdown: from $\lfloor n/2 \rfloor - 1$ starting layer by layer upwards $O(n)$

4.4 Tree Operations

- Tree Storage Structure

- Children: Multi-link list or left child right sibling: Binary link list or parent: Multi-link list
- Tree Traversal
- Pre-order traversal: DFT $O(n)$: Find, output (Generalized list)
- Post-order traversal: $O(n)$: Find depth, clear, find leaf count. Out of queue: visit
- Level-order traversal: BFS $O(n)$ (time): $k-1$ (empty); k : degree; n : depth. Root in queue, left to right in queue

4.5 Tree Applications

- Space Tree

Chapter 3: Graphs

Section 3.1: Graphs and Their Basic Concepts

- **Definition of a Graph:**

$$G = (V, E)$$

Where V is the set of vertices and E is the set of edges. $G' \subseteq G$ represents a subgraph of G .

- **Degree of a Vertex:**

- **Undirected Graph:** The degree of a vertex is the number of edges connected to it.

- **Directed Graph:** The degree of a vertex is the sum of its in-degree and out-degree. The total degree of a graph is twice the number of edges.

- **Path and Connectivity:**

- **Undirected Graph:** A connected graph is one where there is a path between every pair of vertices. A connected component is a maximal connected subgraph.

- **Directed Graph:** A strongly connected graph is one where there is a directed path between every pair of vertices. A strongly connected component is a maximal strongly connected subgraph.

Section 3.2: Forests and Tree Traversal

- **Forest to Binary Tree Conversion:**

- Left child-right sibling

- Binary tree to forest conversion: Left child-right sibling

- **Tree Traversal:**

- **Preorder Traversal:** Root-left-right

- **Postorder Traversal:** Left-right-root

- **Inorder Traversal:** Left-root-right

Section 3.3: Graph Traversal

- **Breadth-First Search (BFS):**

- Level-by-level traversal

- Queue-based implementation

- **Depth-First Search (DFS):**

- Recursive implementation

- Stack-based implementation

Section 3.4: Graph Traversal Algorithms

- **BFS:**

- Queue-based implementation

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

- **DFS:**

- Recursive implementation

- Stack-based implementation

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

Section 3.5: Graph Traversal Applications

- **BFS:**

- Used for finding the shortest path in an unweighted graph.
- Time complexity: $O(V + E)$
- Space complexity: $O(V)$

- **DFS:**

- Used for finding cycles in a graph.
- Time complexity: $O(V + E)$
- Space complexity: $O(V)$

Section 3.6: Graph Traversal Variations

- **BFS Variations:**

- Level-by-level traversal
- Queue-based implementation

- **DFS Variations:**

- Recursive implementation
- Stack-based implementation

Section 3.7: Graph Traversal Optimization

- **BFS Optimization:**

- Use of a queue to manage nodes to be visited.

- Time complexity: $O(V + E)$
- Space complexity: $O(V)$
- **DFS Optimization:**
- Use of a stack to manage nodes to be visited.
- Time complexity: $O(V + E)$
- Space complexity: $O(V)$

Section 3.8: Graph Traversal Applications

- **BFS Applications:**
- Finding the shortest path in an unweighted graph.
- Time complexity: $O(V + E)$
- Space complexity: $O(V)$
- **DFS Applications:**
- Finding cycles in a graph.
- Time complexity: $O(V + E)$
- Space complexity: $O(V)$

Section 3.9: Graph Traversal Challenges

- **BFS Challenges:**
- Time complexity: $O(V + E)$
- Space complexity: $O(V)$
- **DFS Challenges:**

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

Section 3.10: Graph Traversal Conclusion

- **BFS Conclusion:**

- Used for finding the shortest path in an unweighted graph.

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

- **DFS Conclusion:**

- Used for finding cycles in a graph.

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

Section 3.11: Graph Traversal Summary

- **BFS Summary:**

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

- **DFS Summary:**

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

Section 3.12: Graph Traversal Exercises

- **BFS Exercises:**

- Finding the shortest path in an unweighted graph.

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

- **DFS Exercises:**

- Finding cycles in a graph.

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

Section 3.13: Graph Traversal Conclusion

- **BFS Conclusion:**

- Used for finding the shortest path in an unweighted graph.

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

- **DFS Conclusion:**

- Used for finding cycles in a graph.

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

Section 3.14: Graph Traversal Challenges

- **BFS Challenges:**

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

- DFS Challenges:

- Time complexity: $O(V + E)$
- Space complexity: $O(V)$

Section 3.15: Graph Traversal Summary

- BFS Summary:

- Time complexity: $O(V + E)$
- Space complexity: $O(V)$

- DFS Summary:

- Time complexity: $O(V + E)$
- Space complexity: $O(V)$

Section 3.16: Graph Traversal Exercises

- BFS Exercises:

- Finding the shortest path in an unweighted graph.
- Time complexity: $O(V + E)$
- Space complexity: $O(V)$

- DFS Exercises:

- Finding cycles in a graph.
- Time complexity: $O(V + E)$
- Space complexity: $O(V)$

Section 3.17: Graph Traversal Conclusion

- BFS Conclusion:

- Used for finding the shortest path in an unweighted graph.

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

- DFS Conclusion:

- Used for finding cycles in a graph.

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

Section 3.18: Graph Traversal Challenges**- BFS Challenges:**

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

- DFS Challenges:

- Time complexity: $O(V + E)$

- Space complexity: $O(V)$

Section 3.19: Graph Traversal Summary**- BFS Summary:**

- Time complexity: O

5.2 Graph Storage Representation

- Adjacency Matrix Representation
- Degree: In-degree: $2n-1$; Out-degree: $2n-1$
- Adjacency List Representation
- Table header, edge points, out-degree = sum of edge points
- Degree = sum of all points connected to each edge point

5.3 Graph Traversal

- Depth-First Traversal (DFT)
- Adjacency Matrix: $O(n^2)$; Adjacency List: $O(n+e)$; Space: $O(n)$
- Similar to tree's preorder traversal; for each connected component, find the smallest neighbor.
- Breadth-First Traversal (BFT)
- Adjacency Matrix: $O(n^2)$; Adjacency List: $O(n+e)$; Space: $O(n)$
- Similar to tree's level-order traversal (using a queue)

5.4 Minimum Spanning Tree (MST)

- Minimum Tree: A connected graph without cycles ($V(G) = V(G)$): n vertices, $n-1$ edges
- Prim's Algorithm: $O(n^2)$
- Always adds the nearest edge to the minimum spanning tree
- Kruskal's Algorithm: $O(e \log n)$
- Always selects the shortest edge in the edge set and adds it to the minimum spanning tree

5.5 Shortest Path Problem

- Single Source Shortest Path: Dijkstra's Algorithm: $O(n^2)$ (SPF)
- Always adds the nearest vertex to the source in the shortest path tree

Section 2: Shortest Path Algorithm (Floyd Algorithm): $O(n^3)$ Space: $O(n^2)$

Initially, the distance matrix values represent the direct distance between vertices (not passing through other points). After adding each vertex V_k , if $A[i][k] + A[k][j] < A[i][j]$, then update $V_i \rightarrow V_j$'s shortest path. Continue until all vertices are considered. The distance matrix now shows the shortest paths.

5.6 Critical Path

- **Topological Sorting and AOV Network:**
 - Most topological sorting (full sorting); least topological sorting (partial sorting).
 - AOV Network: Directed acyclic graph.
 - Topological sorting: Partial order relationship derived from the full order relationship, not unique; not necessarily from source to target.
- **AOE Network and Critical Path:**
 - AOE Network: Single source, single sink directed acyclic graph.
 - Critical path: The longest path from the source to the sink.
 - Critical activity: Activities on the critical path.
 - Event: Early event $V_i = \max\{V_i + \text{weight}(V_i \rightarrow V_j)\}$
 - Event: Late event $V_i = \min\{V_i - \text{weight}(V_i \rightarrow V_j)\} = V_i - \max\{\text{weight}(V_i \rightarrow V_j)\}$
 - Activity: Early $e < V_i \rightarrow V_j > = V_i$
 - Late $L < V_i \rightarrow V_j > = V_i - \text{weight}(V_i \rightarrow V_j)$
 - If $< V_i \rightarrow V_j > = < V_j \rightarrow V_i >$ forms the critical path, it needs to be accelerated.

Chapter 6: Searching

6.1 Sequential Search

- Sequential Search
- Recursive implementation: while (condition)

6.2 Index Search

- Index Storage Structure:
- Indexing
- Index Table + Subtable
- Index Table + Subtable
- Suitable for sequential tables
- (Pointer Field + Data Field)
- Can use sequential or linked lists for subtables
- Two-level search: Search index ($i = g(K)$, K as key, find subtable) → Search subtable (linked list)
- Multi-index (index table too large): Search index table again
- Block Search:
- Index table is an ordered table, index table key is the maximum value of each subtable element, subtables are ordered, tables are not
- m : Index table length, n : Main table length (divided into m subtables), time: $O(\log(m+1) + n/m)$ or $O(n) + O(m)$
- Index File:
- Index File (Main File): Index item (key) with file record corresponding to: Dense Index
- Index Table: Physical record with file record corresponding to a group of records: Sparse Index
- If index table is too large, rebuild index table → Search table, search item corresponds to a group of index items

- Insert operation:
- Insert record into file tail (last block, need to split a new block)
- Insert corresponding index item into index table: $i = g(K)$
- For index sequential file:
- In main file, search for insertion position, insert record
- Modify index table (sparse index), insert index item into index table (dense index)

Delete operation:

- In main file, add "delete" mark to deleted record
- In index table, delete corresponding record in index table (sparse index) (not done)

Notes on Data Structures and Algorithms

6.3 Hash Table Searching

- **Linear Hash Table:** Calculates $i = H(k)$. If empty, insert; otherwise, search sequentially. $i = i + 1 \mod M$. Cannot solve overflow issues; collision frequency is the load factor.
- **Random Hash Table:** Calculates $i = H(k)$. If not empty, $i = i_0 + RN$, where i_0 is the initial position and RN is a random number. Not suitable for frequent insertions and deletions.
- **External Link Hash Table:** Calculates $i = H(k)$. Search the linked list sequentially (linked list search). Requires more external space, increasing time complexity.
- **Overflow Hash Table:** Calculates $i = H(k)$. If not empty, search the overflow table (linear probing).

6.4 Tree Searching

- **Binary Search Tree vs. AVL Tree:** Dynamic search vs. static search.
- **AVL Tree:** Height h , $n \geq F(h + 2) - 1$. $F(0) = F(1) = 1, \leq 2^h - 1$.
- **Rotation Operations:**
- **Left-Left (LL):** Root A is deeper than its left child P , and P is deeper than its left child.
- **Right-Right (RR):** Root A is deeper than its right child P , and P is deeper than its right child.
- **Left-Right (LR):** Root A is deeper than its right child P , and P is deeper than its left child. Perform a right rotation on P and a left rotation on A .

- **Right-Left (RL):** Root A is deeper than its left child P , and P is deeper than its right child. Perform a left rotation on P and a right rotation on A.

Insertion and deletion operations are performed by rotating the tree.

III. B-Tree

A B-tree of order m is a self-balancing search tree that satisfies the following properties:

- Every node has at most m children.
- Every non-leaf node (except the root) has at least $\lceil m/2 \rceil$ children.
- The root has at least 2 children if it is not a leaf node.
- A non-leaf node with k children contains $k - 1$ keys.
- All leaves appear on the same level.

Chapter 7 Sorting

7.1 Sorting by Exchange

- Bubble Sort
- Each pass reverses the array, moving the largest element to the end; record the last position of the move to improve efficiency.
- Comparison:

$$\frac{1}{2}n(n-1)$$

,

$$O(n^2)$$

, stable, consider bidirectional bubble

- Quick Sort
- Divide and conquer: Each time select a pivot to split the array into two parts, until fully sorted. Select $R[n]$ as the pivot.
- Comparison:

$$O(n \log n)$$

~

$$O(n^2)$$

, movement:

$$O(n)$$

~

$$O(n)$$

, unstable, suitable for large arrays

7.2 Insertion Sort

- Direct Insertion Sort
- Each element is inserted into the sorted part, sequentially searching the sorted part.
- Comparison: $n-1$, movement: $2n-1$,

$$O(n^2)$$

, stable, suitable for small basic sorted arrays

- Binary Insertion Sort
- Binary search in the sorted part.
- Comparison:

$$O(n \log n)$$

, movement:

$$O(n)$$

,

$$O(n^2)$$

, stable

- Binary Insertion Sort
- Use binary insertion method to insert elements of the unordered part into the front or rear part of the sorted part.
- Comparison:

$$O(n \log n)$$

, movement:

$$O(n)$$

- Shell Sort
- Divide and conquer: Divide the array into subsequences based on the gap sequence, and sort each subsequence.
- Comparison:

$$O(n^{\frac{5}{2}})$$

, unstable

7.3 Selection Sort

- Simple Selection Sort
- Each pass selects the largest element from the unordered part and places it at the end of the sorted part.
- Comparison:

$$\frac{1}{2}n(n-1)$$

, movement: $3(n-1)$,

$$O(n^2)$$

, unstable

Two. Heap Sort

Tree Selection Sort:

Heap Sort:

Heap Sort:

7.4. Merge Sort

1. Merge Sort Concept:

Merge (Merge):

Merge Sort:

Two. Merge of Adjacent Ordered Subarrays:

Three. Merge Sort Implementation:

7.5. External Sorting

1. External Sorting Basic Steps:

Two. External Sorting Time Complexity:

Huffman Tree and Optimal Merge Tree

Huffman Tree:

- **Definition:** A Huffman tree is a binary tree where the left child is less than the right child; all Huffman trees reduce the number of moves.
- **Comparison:** Similar to the merge tree sorting method (heap sort).

Optimal Merge Tree:

- **Number of Merge Segments:** $\log_k \frac{n}{T}$
- **Time Complexity:** $O(n \log_k \frac{n}{T})$
- **Optimal Merge Tree:** From the initial merge segment length as the value of the Huffman tree:
- **Degree of Leaf Nodes:** $M_0 = (k - 1)M_k + 1$
- **When $M_0 - 1$ is not an integer multiple of $k - 1$,** add virtual merge segments and fill leaf nodes. $\Rightarrow$ Get the order of all merge segments for merging.

Required Algorithm

1. Linear List Insertion (Circular List Doesn't Need a Head)

- Allocate a new node
- Set the new node's value
- Handle the list
- If element b is at the first position
- Search for element b from the second position
- If found, insert a before b; otherwise, append to the end of the list

2. Linear List Deletion (Similar to Insertion)

- Search for element b in the list
- If not found, delete fails; handle b in the first position
- Otherwise, perform normal deletion; release the deleted node

3. Linear List Merge (Circular List Must Include New List Tail)

- Handle the list situation
- Take X as the basic list
- When X and Y lists have unmerged nodes, connect Y's nodes to X's nodes
- If X has unmerged nodes, connect X's nodes to Y's nodes

4. Linear List Split (Circular List: Take X as the basic list, split Y out)

- Divide into odd and even

II. Binary Tree Traversal Algorithm

- Depth-first (post-order traversal)
- Sequence: BTreeNode *p;
- if (BT == NULL) return 0;
- else if (BT != NULL) EnQueue(BT);
- int dep1 = depth(BT->left);
- int dep2 = depth(BT->right);
- p = OutQueue();
- if (dep1 > dep2) return dep1 + 1;
- else return dep2 + 1;

III. Binary Search Tree Construction Algorithm, In-order Traversal Algorithm

- Build Binary Search Tree Algorithm

```

void thread(BT)
{
    static Node *pre = NULL;
    if (BT != NULL)
    {
        thread(BT->lchild);
        if (pre != NULL && pre->rtag == 1) // current thread's left child needs
            pre->rchild = BT;
        if (BT->lchild == NULL) // current thread's left child is null
            BT->ltag = 1; BT->lchild = pre;
        if (BT->rchild == NULL) // current thread's right child is null
            BT->rtag = 1;
        pre = BT; // current thread is the parent
        thread(BT->rchild);
    }
}

```

2. Utilize threads in a binary tree for inorder traversal

```

Node *p = BT; // initial pointer to root
if (p != NULL)
{
    while (p->ltag == 0) p = p->lchild; // find the inorder predecessor
    do
    {
        if (p->rtag == 1) p = p->rchild; // is a thread, move to the inorder successor
        else
        {
            p = p->lchild; // is a child, move to the inorder predecessor
        }
    }
}

```

```
    }  
    } while (p->ltag == 0) p = p->lchild; // find the inorder predecessor  
}
```

The provided code snippet is a C++ function `thread` that threads a binary tree. It traverses the tree and threads it in such a way that each node has a thread to its inorder predecessor. The second part of the code snippet is a pseudocode for inorder traversal using threads. It starts at the root and moves left until it finds a node with a left thread, then it moves right until it finds a node without a right thread, then it moves left again until it finds a node with a left thread, and so on.

Section 1: Binary Search Recursive and Non-Recursive Algorithms

- **Recursive Implementation:**

- If $\text{left} \leq \text{right}$, take the middle point.
- If middle point value $<$ search value, middle point = recursive right half (middle + 1).
- If middle point value $>$ search value, middle point = recursive left half (middle - 1).
- If middle point value = search value, stop.

- **Non-Recursive Implementation:**

- If $\text{left} \leq \text{right}$, take the middle point.
- If middle point value $<$ search value, $\text{left} = \text{middle} + 1$.
- If middle point value $>$ search value, $\text{right} = \text{middle} - 1$.
- If middle point value = search value, stop.

Section 2: Simple Matching Algorithm and KMP Algorithm

- **Simple Matching:**

- Outer loop: check if the pattern matches.
- Inner loop: move the pattern.

- **KMP:**

- `int m = strlen(P); n[0] = 0;`
- `int i = 1, j = 0; // Initialize matching position.`
- `while (i < m) {`
- `if (pl[i] == pl[j]) {`

- $n[i] = j + 1$; // Partial matching length increases.
- $i++$; $j++$;
- } else if ($j > 0$) {
- $j = n[j - 1]$;
- } else {
- $n[i] = 0$; // j at the beginning, partial matching length is 0.
- }
- }

```
int kmp_match(text T, pattern P)
{
    int n = strlen(T), m = strlen(P);
    int i = 0, j = 0, nxt[MaxSize];
    Next(P, nxt);
    while (i < n)
    {
        if (T[i] == P[j])
        {
            if (j == m - 1) return i - j; // match success
            else { i++; j++; }
        }
        else
        {
            if (j > 0) j = nxt[j - 1]; // move to partial match point
            else { i++; }
        }
    }
    return -1;
}
```

VI. Sorting Algorithms

1. Bubble Sort

```
void bubble(a left right)
{
    for (int i = left; i < right; i++)
        for (int j = right; j > i; j--)
            if (a[j] < a[j - 1]) swap(a[j], a[j - 1]);
}
```

```
}
```

Improvement: flag records the position of the last exchange in a pass of compa

- Quick Sort

Partition Function:

```
int Partition(a, left, right)
```

```
{
```

```
int k = left;
```

```
int i = left, j = right;
```

```
while (i != j) // when i and j meet, this pass of partitioning is complete
```

```
{
```

```
while (a[j] >= a[k] && i < j) j--; // move j left until a[j] < a[k]
```

```
if (i < j) // a[j] < a[k]
```

```
{
```

```
a[i] = a[j]; i++;
```

```
// a[i] and a[j] swap
```

```
}
```

```
while (a[i] <= a[k] && i < j) i++; // move i right until a[i] > a[k]
```

```
if (i < j) // a[i] > a[k]
```

```
{
```

```
a[i] = a[j]; j--;
```

```
// a[i] and a[j] swap
```

```
}
```

```
}
```

```
a[i] = a[k]; // insert a[k] at partitioning point i
```

```
return i; // return this pass's partitioning point i
```

```
}
```

Quick Sort:

```
void QuickSort(a, left, right)
```

```
{
```

```
int i;
```

```
if (right > left) // table length is not less than 2
```

```
{
```

```
i = Partition(a, left, right); // sequence partitioning operation
```

```
QuickSort(a, left, i - 1); // quicksort left side
```

```
QuickSort(a, i + 1, right); // quicksort right side
```

```
}
```

```
}
```

- Insert Sort:

```
void Insert(...) // Direct Insertion
{
    for (int j = 1; j < len; j++) // Need len-1 insertions
    {
        int temp = a[j]; int i = j - 1; // Store the first element of the s
        while (i >= 0 && a[i] > temp) // i points to the last position of t
        {
            a[i + 1] = a[i]; i--;
        } // Move temp to its insertion position
        a[i + 1] = temp; // Insert the element to be inserted into the sort
    }
}
```

- Binary Insert Sort:

```
void BInsert(...) // Binary Insertion
{
    for (int j = 1; j < len; j++)
    {
        int temp = a[j]; int low = 0, high = j - 1;
        while (low <= high) // Binary search for the insertion element in t
        {
            int mid = (low + high) / 2;
```

```
        if (a[mid] > temp) high = mid - 1;
        else low = mid + 1;
    }
    for (int i = j - 1; i <= high + 1; i--) // Shift elements from high
    {
        a[i + 1] = a[i];
    }
    a[high + 1] = temp;
}
}
```

- Shell Sort

```
void Shell(a, times, d[]){ // times: sort intervals, d[]: gap between intervals
    for(int k = times; k > 1; k--) {
        for(int j = k; j < len; j++) {
            int h = d[k];
            for(int i = j - h; i >= 0; i -= h) {
                int temp = a[i];
                a[i] = a[i + h];
                a[i + h] = temp;
            }
        }
    }
}
```

- Merge Sort

```
void MergeSort(a) {
    int *work = new int[len];
    int length = 1; // initial subarray length
    while(length < len) {
        int cur = 2 * length; // cur: current merged subarray length
        for(int t = 0; t < len; t += cur) {
            int low = t, high = t + cur - 1, mid = t + length - 1;
            if(high > len - 1) high = len - 1; // special handling for the last
```

```
        if(high > mid) merge(a, low, mid, high, work); // merge subarrays
        length = cur;
    }
}
delete[] work;
}
```

- **Top Down:**

```
for(int m = 1; m <= n / 2; m = 2 * m)
    merge_sort(a, left, (left + right) / 2);
```

- **Bottom Up:**

```
for(int i = left; i <= right - m; i += m)
    merge_sort(a, i, i + m, min(i + 2 * m - 1, right));
```

- Selection Sort

```
void SelectSort(a) {  
    for (int i = 0; i < len - 1; i++) {  
        int pos = i;  
        for (int j = i + 1; j < len; j++) {  
            if (a[j] < a[pos]) {  
                pos = j;  
            }  
        }  
        if (pos != i) {  
            swap(a[i], a[pos]);  
        }  
    }  
}
```

- Heap Sort

```
void heapSort(a--) {  
    int len = right - left; int *p = a + left;  
    for (int k = (len - 1) / 2; k >= 0; k--) {  
        SiftDown(p, k, len);  
    }  
    while (len > 0) {  
        exchange(p[0], p[len]);  
    }  
}
```

```
        len--;  
        SiftDown(p, 0, len);  
    }  
}
```

The code provided is for implementing selection sort and heap sort algorithms in C.

- Merge Sort (Continued).

```
void merge(int *c, int *a, int n, int *b, int m) {
    long i, j, k;
    for (i = 0, j = 0, k = 0; k < n + m; k++) {
        if (i == n) { c[k] = a[j]; j++; continue; }
        if (j == m) { c[k] = b[i]; i++; continue; }
        c[k] = (a[i] < b[j]) ? a[i++] : b[j++];
    }
}
```

Bottom-Up Merge Sort:

```
void merge_sort(int *a, int left, int right) {
    for (int i = 1; i <= right - left; i = 2 * i) {
        for (int j = left; j <= right - i; j += 2 * i) {
            merge(a, j, j + i, min(j + 2 * i - 1, right), a[j]);
        }
    }
}
```

Top-Down Merge Sort:

```
void merge_sort(int *a, int left, int right) {  
    if (right <= left) return;  
    int mid = (left + right) / 2;  
    merge_sort(a, left, mid);  
    merge_sort(a, mid + 1, right);  
    merge(a, left, mid + 1, right);  
}
```

Time complexity: $O(n \log n)$

Space complexity: $O(n)$